

The Collaborative Coder's Guide: Mastering the Pull Request Workflow

In modern software development, working directly on a single codebase is like multiple people trying to paint on the same canvas at the same time—eventually, someone will paint over someone else's hard work. To build stable, professional applications, we move away from "cowboy coding" and toward a structured journey called the **Pull Request (PR) Workflow**.

1. The "Why" Behind the Workflow: Safety in Isolation

Before we touch the code, you must understand the "Single-Branch Approach" risk. When every developer pushes to main (or master) simultaneously, work is frequently overwritten or lost. In a professional environment, main represents a "perfect snapshot" of a stable project. We protect that snapshot by working in "little timelines" called feature branches.

Feature	Single-Branch (Master-Only) Approach	Feature Branch Approach
Risk Level	High: Dev B can overwrite Dev A's work 5 seconds later.	Low: Work happens in a "Private Sandbox."
Stability	Deployment breaks easily due to untested, direct code.	Main remains stable; only "Green Lit" code enters.
Collaboration	Developers step on each other's toes and wait for turns.	Everyone works in their own lane simultaneously.
Code Quality	Errors slip through via direct pushes.	Peer reviews catch bugs before they reach the "Castle."
Recovery	Difficult: Overwritten work is often "gone."	Easy: You have isolated timelines that can be restored.

Instructor Note: In the industry, a broken main branch means a broken deployment. Isolation isn't just a preference; it's a requirement for professional survival.

Transition: Now that your environment is safely isolated, we can begin the iterative process of development without fear of disrupting the main codebase.

2. Step 1: Branch Creation and Naming Conventions

The workflow begins in **GitHub Desktop** (or your CLI) by creating a new branch. This acts as your private sandbox.

- **The Power of Prefixes:** Use a naming convention like feature/ or bugfix/ (e.g., feature/add-splash-screen).
 - **Team Benefit:** This creates logical "folders" in the GitHub web interface. It makes it significantly easier for your team to navigate the repository and see who is working on what.
- **The "Publish" Requirement:** After creating the branch locally, you **must** click **Publish**.

- **Common Pitfall:** Until you "Publish," your teammates cannot see your branch, your progress, or your Pull Request. It doesn't exist to the remote repository until you push it live.

Transition: Once the branch is live and the sandbox is ready, it is time to shift to your IDE and get to work.

3. Step 2: The Development Loop (Code, Commit, Push)

Development happens in a loop within **Android Studio**. This is where your feature takes shape.

1. **Code:** Implement your changes. Because you are in your own branch, you cannot break the live project.
2. **Commit:** Save your changes locally.
 - **Pro Tip:** Use the integrated AI tools in your Git window to generate commit summaries. This ensures your work is documented for the team. Don't be the developer who just writes "I added something"—professional maturity means providing context.
3. **Push:** Send your local commits to the GitHub server.
 - **Safety Check:** These changes live **only** on your feature branch. The main branch remains untouched.

Transition: Once the code is pushed and verified on your end, the process moves from solo work to team collaboration.

4. Step 3: The Pull Request (PR) Lifecycle

A Pull Request is a formal proposal to merge your changes into the stable codebase.

- **Phase A: Creation & Description:** On GitHub, you document **what** was changed and **why**. This is a critical safety check: run your **automated tests** here. You need to ensure the code works "online" in the cloud environment, not just on your local machine.
- **Phase B: Assignment:** Assign a peer (like a teammate or "Jessica" from our example) as a **Reviewer**. This turns the merge into a collective decision rather than a solo risk.
- **Phase C: The Review:** Your reviewer examines the code and takes one of three actions:

The Reviewer's Toolkit

- **Comment:** Provide feedback or ask questions without blocking the merge.
- **Request Changes: Block the merge** until specific issues or bugs are resolved.
- **Approve:** Give the "green light" that the code is ready for the main branch.

Transition: This phase ensures collective agreement before the code is allowed to enter the "Castle."

5. Step 4: Protecting the Castle (Branch Protection Rules)

To safeguard your project, you must configure your repository to prevent accidents. In **GitHub Settings > Rules**, you can establish "Rule Sets" to "Protect the Castle."

1. **Restrict Direct Pushes to Main:** This forces every single change to go through a PR.
2. **Require Pull Request Approvals:** The system will physically block a merge until a teammate (the reviewer) has hit "Approve."
3. **Rejecting "Force Pushes":** While many students rely on "Force Pushing" to fix mistakes, it is a dangerous habit that can delete others' work. In a professional workflow, if you follow this pattern correctly, you will never need to "force" anything.

Transition: Once the protection requirements are met and the peer review is complete, the code is ready for its final destination.

6. Step 5: The Merge and Branch Cleanup

The final integration happens through the **Merge**. This takes your tested, reviewed code and integrates it into the stable main timeline.

Crucial Housekeeping: No Branch Hoarding Feature branches are **short-lived**. Unlike main or production (which are "Long-Lived"), a feature branch should be deleted the moment it is merged. **There is no point in referring back to these branches again** once the code is in main. Delete the branch on GitHub and locally in GitHub Desktop to keep your workspace clean.

Transition: Even with these rules, you must be prepared for the moment when two developers' timelines collide.

7. Troubleshooting: Navigating Merge Conflicts

A **Merge Conflict** is "guaranteed" to happen in team projects. It occurs when Dev A and Dev B edit the same line of code.

The "Outdated" Scenario: If Dev A merges their changes to main while Dev B is still working, Dev B's code is now "outdated." When Dev B tries to merge, GitHub will **block** the action because the system doesn't know which version of the line is correct.

How to Resolve:

1. **Identification:** GitHub (or Android Studio) will flag the specific files and lines where the conflict exists.
 2. **Manual Choice:** You must manually choose which version to keep—yours, theirs, or a hybrid of both.
 3. **Commit the Fix:** Once resolved, you commit the "resolved" version to complete the merge.
-

8. Quick-Reference Workflow Checklist

- [] **Create Branch:** Use a naming convention (e.g., feature/description).
- [] **Publish Branch:** Make the branch visible to your teammates on GitHub.
- [] **Development Loop:** Code in Android Studio, use AI to generate detailed commit messages, and push to your branch.
- [] **Open Pull Request:** Document your "Why," run automated tests, and assign a reviewer.
- [] **Peer Review:** Address any **Request Changes** or **Comments** from your team.
- [] **Merge:** Once you have the "Approve" green light, merge into the main branch.
- [] **Cleanup:** Immediately **Delete** the short-lived feature branch locally and on GitHub.

The Collaborative Workflow: Mastering GitHub Feature Branches

Feature Branching prevents overwriting. Isolate work, collective approval via Pull Requests, delete temporary branches for a clean repo.

The Sandbox Strategy



THE CASTLE (Main Branch)

A perfect, stable snapshot of the current project. Must be protected from broken code.



SANDBOXES (Feature Branches)



Feature Branch Isolation:
Private sandbox for specific features without affecting the live project.



**KEY FINDING:
The Single-File Danger.**
Overwriting: Dev B's save deletes Dev A's work 5 seconds later.



The Request & Review:
Explain changes & assign peer reviewers, instead of forcing into Main.



Collective Approval:
Code is only merged after teammate review and approval, ensuring group agreement.



Merge & Cleanup:
Feature branch deleted immediately to prevent "long-lived" branch clutter.

Branch Lifespan Comparison

Branch Type	Lifespan	Purpose
Main / Master	Permanent	The stable, live production code.
Feature / Bugfix	Temporary	Private lane for coding and testing.
Merged Branch	Deleted	